

1. With a neat diagram, discuss the split view of the kernel in detail.

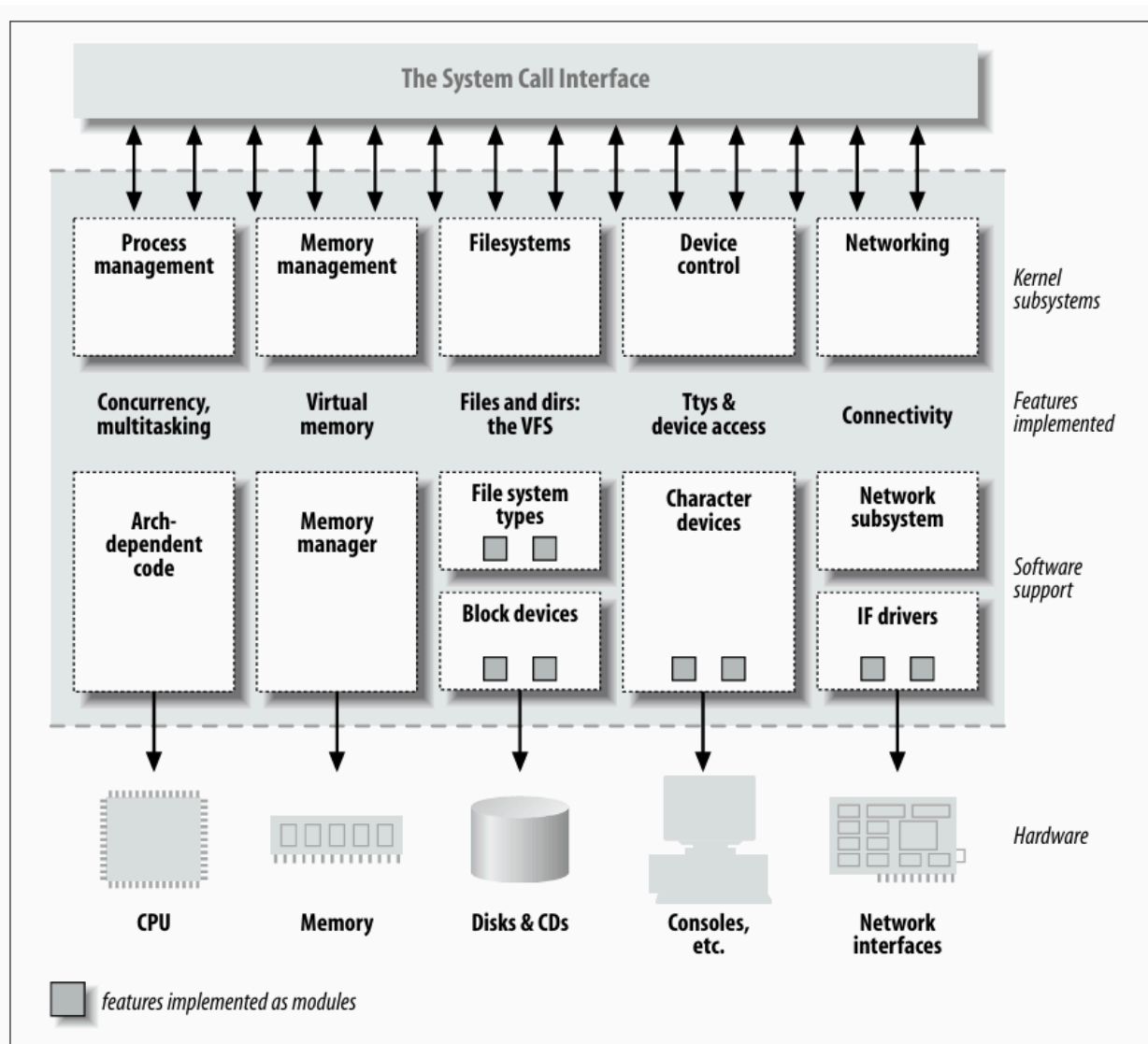


Figure 1-1. A split view of the kernel

1. Process Management
2. Memory Management
3. Filesystems
4. Device Control
5. Networking

The kernel is in charge of creating and destroying processes and handling their connection to the outside world (input and output). Communication among different processes (through signals, pipes, or interprocess communication primitives) is basic to the overall system functionality and is also handled by

the kernel. In addition, the scheduler, which controls how processes share the CPU, is part of process management.

The computer's memory is a major resource, and the policy used to deal with it is a critical one for system performance. The kernel builds up a virtual addressing space for any and all processes on top of the limited available resources. The different parts of the kernel interact with the memory-management subsystem through a set of function calls, ranging from the simple malloc/free pair to much more complex functionalities.

Unix is heavily based on the filesystem concept; almost everything in Unix can be treated as a file. The kernel builds a structured filesystem on top of unstructured hardware, and the resulting file abstraction is heavily used throughout the whole system. In addition, Linux supports multiple filesystem types, that is, different ways of organizing data on the physical medium

Almost every system operation eventually maps to a physical device. With the exception of the processor, memory, and a very few other entities, any and all device control operations are performed by code that is specific to the device being addressed. That code is called a device driver.

The OS must handle networking because incoming packets arrive asynchronously and need to be collected, identified, and routed before any process uses them. The system is in charge of delivering data packets across program and network interfaces, and it must control the execution of programs according to their network activity. Additionally, all the routing and address resolution issues are implemented within the kernel.

2. Discuss the role of a device driver. How to load modules in the kernel subsystem.

The role of a device driver is providing mechanism, not policy.

Mechanism to access hardware.

Most programming problems can indeed be split into two parts: "what capabilities are to be provided" (the mechanism) and "how those capabilities can be used" (the policy). If the two issues are addressed by different parts of the program, or even by different programs altogether, the software package is much easier to develop and to adapt to particular needs.

write kernel code to access the hardware, but don't force particular policies on the user, since different users have different needs. The driver should deal with making the hardware available, leaving all the issues about how to use the hardware to the applications.

E.g., Hard disk

Read/write blocks of data

Access as a continuous array

Policy-Free Implementation

Simplifies the design

Reuse: Different policies do not require changes to the kernel

Using insmod:

insmod loads a module into the running kernel. Returns the value of the module's init function. It performs the following steps:

- Copies the module's code and data into kernel memory
- Resolves any **unresolved symbols** using the kernel's symbol table

Calls the module's **initialization function** (**module_init** routine). Internally, this uses the system call: `sys_init_module()`

Ex:

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");
static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}
static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

```
% make
make[1]: Entering directory `/usr/src/linux-2.6.10'
CC [M] /home/ldd3/src/misc-modules/hello.o
Building modules, stage 2.
MODPOST
CC /home/ldd3/src/misc-modules/hello.mod.o
LD [M] /home/ldd3/src/misc-modules/hello.ko
make[1]: Leaving directory `/usr/src/linux-2.6.10'
% su
root# insmod ./hello.ko
Hello, world
root# rmmod hello
Goodbye cruel world
root#
```

Using modprob:

modprobe also loads modules but is **smarter** than insmod.

It:

- Checks if the module has unresolved symbols
- Automatically loads **dependency modules**
- Loads the requested module and all modules it depends on

If you used `insmod` instead, the command would fail with “**unresolved symbols**”.

Using `kmod + request_module()`:

This is the most important kernel-side mechanism.

When the kernel needs a resource (a driver, filesystem, network protocol, etc.) and it is not present, it calls:

`request_module("module_name");`

- The `kmod` subsystem (enabled in kernel config) then spawns a kernel thread that runs the userspace program `modprobe`.
- Kernel requests are handled by `kmod` instead of returning errors.
- `request_module()` loads by name or by “capability.”
- The call is synchronous—it blocks until loading is attempted.
- The userspace “helper” (`modprobe`) is run by the kernel thread.

Loading Modules at Boot via Init Scripts / System Config

Modules can be loaded automatically at boot, using:

1. `/etc/modules.conf` or `modprobe.conf` directives

Example:

```
alias eth0 eeepro100
```

```
pre-install pcmcia_core /etc/rc.d/init.d/pcmcia start
```

```
options short irq=1
```

These instruct the system to load modules early in boot.

2. Init scripts (e.g., `rc.local`)

The driver can be loaded via:

```
modprobe mydriver
```

The textbook explicitly mentions using init scripts to load modules.

Loading Modules by the Filesystem or Hardware Detection

When systems detect new hardware—or when filesystems are mounted—they may trigger module loads via `kmod`.

Examples in the book:

- Network drivers (e.g., `eth0`) are requested through `ifconfig`.
- Filesystem modules are loaded when a filesystem type is requested.

3. Discuss the commands to load and remove the modules in a kernel.

Using `insmod`:

`insmod` loads a module into the running kernel. Returns the value of the module’s init function.

It performs the following steps:

- Copies the module’s code and data into kernel memory

- Resolves any **unresolved symbols** using the kernel's symbol table

Calls the module's **initialization function** (`module_init` routine)

Internally, this uses the system call:

`sys_init_module()`

`insmod` failure:

Unknown/unfound symbol

Dependent modules are not yet loaded

Return value of `module_init` is non-zero

`modprobe` also loads modules but is **smarter** than `insmod`.

It:

- Checks if the module has unresolved symbols
- Automatically loads **dependency modules**
- Loads the requested module and all modules it depends on

If you used `insmod` instead, the command would fail with “**unresolved symbols**”.

Removal;

`rmmod` removes a module from the kernel.

Removal fails if:

- The module is **in use** (e.g., an open device file exists)
- The kernel is configured to disallow removal

Forced removal is possible but dangerous—reboot is safer.

`lsmod` lists currently loaded kernel modules.

It reads the virtual file: `/proc/modules` or `/sys/module`

`modprobe -r` — Safe Removal with Dependency Handling

Purpose:

Removes a module and any modules that depend on it.

Usage:

`sudo modprobe -r module_name`

Advantages:

- Reads dependency information to prevent unsafe removal.

- Supports pre-remove and post-remove commands in /etc/modules.conf.

4. Represent the sequence of operations performed while the process performs read() and write() implementation in a scull environment.

Code present in pages 91-94 of Linux device drivers.

Short summary of steps:

READ() Sequence

1. User calls read().
2. VFS calls scull_read().
3. Identify position → quantum set → quantum.
4. Check EOF and compute readable bytes.
5. copy_to_user() moves data to user buffer.
6. Update f_pos.
7. Return number of bytes read.

WRITE() Sequence

1. User calls write().
2. VFS calls scull_write().
3. Identify position → quantum set → quantum.
4. Allocate memory if necessary.
5. copy_from_user() moves data into SCULL memory.
6. Update f_pos and device size.
7. Return number of bytes written.

5. List and write any 10 fields in file structures of a scull environment.

struct file, defined in <linux/fs.h>

Allocated when a file/device is opened

ref count incremented when new references are created

freed on "last close" of a file/device

A pointer to file is often called filp.

mode_t f_mode:

The file mode identifies the file as either readable or writable (or both), by means of the bits

FMODE_READ and FMODE_WRITE.

loff_t f_pos:

The current reading or writing position. loff_t is a 64-bit value on all platforms

unsigned int f_flags:

These are the file flags, such as O_RDONLY, O_NONBLOCK, and O_SYNC.

All the flags are defined in the header <linux/fcntl.h>

struct file_operations *f_op:

The operations associated with the file.

We can change `filp->f_op` during `open()`, and the kernel will immediately start using the new file-operation methods, allowing different behaviors under one driver—like method overriding in OOP

void *private_data:

Can be used to store additional data structures

Needs to be freed during the release method

struct dentry *f_dentry

Directory entry associated with the file

Used to access the inode data structure

`filp->f_dentry->d_inode`

struct fown_struct f_owner:

Stores information about the “owner” of the file.

Used when sending **SIGIO / SIGURG** asynchronous notifications to a process.

Important in drivers that use asynchronous I/O.

struct path f_path:

Contains the full path (dentry + mount) of the opened file.

Accessed using: `filp->f_path.dentry`.

unsigned long f_version:

Version number of the file.

Updated on each write operation.

struct file_ra_state f_ra:

Stores the **read-ahead information** for the file.

The kernel uses this to **predict future reads** and load data in advance to improve performance.

When you read a file sequentially, the kernel assumes you will read the next part soon.

So it **preloads extra data into the page cache** *before* your program asks for it.

6. List and discuss the configuration options that should be enabled for kernels used for development during the debugging process.

We list the configuration options that should be enabled for kernels used for development.

all of these options are found under the “kernel hacking” menu. Some of the options are not supported by all architectures.

CONFIG_DEBUG_KERNEL

This option just makes other debugging options available; it should be turned on but does not, by itself, enable any features.

CONFIG_DEBUG_SLAB

This helps detect memory errors.

Each byte of allocated memory is set to 0xa5 before being handed to the caller and then set to 0xb6 when it is freed. If you ever see either of those “poison” patterns repeating in output from your driver (or often in an oops listing), you’ll know exactly what sort of error to look for. When debugging is enabled, the kernel also places special guard values before and after every allocated memory object; if those values ever get changed, the kernel knows that somebody has overrun a memory allocation, and it complains loudly

CONFIG_DEBUG_PAGEALLOC

When a page is freed, the kernel **removes it from address space**.

If your code touches it again → instant error.

Useful for finding memory corruption.

CONFIG_DEBUG_SPINLOCK

Checks for **wrong spinlock usage**, such as:

- Using an **uninitialized** spinlock
- Unlocking twice
- Unlocking without locking

Helps catch synchronization bugs.

CONFIG_DEBUG_SPINLOCK_SLEEP

Warns if code tries to **sleep while holding a spinlock**.

Sleeping with a spinlock is a serious bug.

CONFIG_DEBUG_MUTEXES

Detects and reports various mutex violations

CONFIG_DEBUG_INFO

Enables gdb debugging

CONFIG_LOCKUP_DETECTOR

Detect hard and soft lockups

Softlockups – cause kernel to loop for more than 60 seconds

Hardlockups – cause cpu (or core) to loop for more than 60 seconds

CONFIG_KGDB*

Remotely debug the kernel using gdb

CONFIG_DEBUG_ATOMIC_SLEEP

Reporting if calling a routine that may sleep inside a critical section

CONFIG_MAGIC_SYSRQ

Enables the **Magic SysRq Key**, a special key combo used when the system hangs.

Very useful for crash recovery.

CONFIG_DEBUG_STACKOVERFLOW

Helps track down kernel stack overflows

CONFIG_DEBUG_STACK_USAGE

Monitors stack usage and makes statistics available via magic SysRq key

CONFIG_KALLSYMS

CONFIG_KALLSYMS stores kernel symbol names so crash messages show readable function names instead of raw addresses. Enable by default.

CONFIG_PROFILING

Allows kernel performance profiling using tools like readprofile.

Can help detect hangs and performance problems.

7. Discuss the different loglevels used in printk() statement.

printk lets us classify messages according to their severity by associating different loglevels, or priorities, with the messages. We usually indicate the loglevel with a macro. The loglevel macro expands to a string, which is concatenated to the message text at compile time.

Ex: `printk(KERN_DEBUG "Here I am %s:%i\n", __FILE__, __LINE__);`

8 possible loglevels defined in <linux/kern_levels.h> (ACEE WIND - shortcut to remember)

KERN_EMERG: For Emergency message, usually message that precedes crash

KERN_INFO: Informational message. Many device drivers print information about the hardware they find at the startup at this level.

KERN_DEBUG: Used for debugging messages.

KERN_WARNING: Warning for less serious problems

KERN_CRIT: Critical Condition. Often related to hardware or software failure.

KERN_ERR: Used to report error conditions. Device drivers often use KERN_ERR to report hardware difficulties.

KERN_ALERT: A situation requiring immediate action.

KERN_NOTICE: Situations that are normal. But still worthy to note. A lot of security related conditions reported at this level.

8. Show the implementation of the semaphore mechanism to avoid race conditions while accessing the scull_dev data structures. Pg 130

The semaphore mechanism gives scull a tool that can be used to avoid race conditions while accessing the scull_dev data structure. But it is up to us to use that tool correctly. The keys to proper use of locking primitives are to specify exactly which resources are to be protected and to make sure that every access to those resources uses the proper locking.

```
struct scull_dev {
    struct scull_qset *data;
    int quantum;
    int qset;
    unsigned long size;
    unsigned int access_key;
    struct semaphore sem; /* mutual exclusion semaphore */
    struct cdev cdev;
};
```

Toward the bottom of the structure is a member called sem which is, of course, our semaphore. We have chosen to use a separate semaphore for each virtual scull device. It would have been equally correct to use a single, global semaphore. The various scull devices share no resources in common, however, and there is no reason to make one process wait while another process is working with a different scull device. Using a separate semaphore for each device allows operations on different devices to proceed in parallel and, therefore, improve performance.

Semaphores must be initialized before use. scull performs this initialization at load time in this loop:

```
for (i = 0; i < scull_nr_devs; i++) {
    scull_devices[i].quantum = scull_quantum;
    scull_devices[i].qset = scull_qset;
    init_MUTEX(&scull_devices[i].sem);
    scull_setup_cdev(&scull_devices[i], i);
}
```

Note that the semaphore must be initialized before the scull device is made available to the rest of the system. Therefore, init_MUTEX is called before scull_setup_cdev. Performing these operations in the opposite order would create a race condition where the semaphore could be accessed before it is ready.

Next, we must go through the code and make sure that no accesses to the scull_dev

data structures are made without holding the semaphore. Thus, for example, `scull_write` begins with this code:

```
if (down_interruptible(&dev->sem))
    return -ERESTARTSYS;
```

Note the check on the return value of `down_interruptible`; if it returns nonzero, the operation was interrupted. The usual thing to do in this situation is to return `-ERESTARTSYS`. Upon seeing this return code, the higher layers of the kernel will either restart the call from the beginning or return the error to the user. If you return `-ERESTARTSYS`, you must first undo any user-visible changes that might have been made, so that the right thing happens when the system call is retried. If you cannot undo things in this manner, you should return `-EINTR` instead. `scull_write` must release the semaphore whether or not it was able to carry out its other tasks successfully. If all goes well, execution falls into the final few lines of the function: `out: up(&dev->sem); return retval;` This code frees the semaphore and returns whatever status is called for. There are several places in `scull_write` where things can go wrong; these include memory allocation failures or a fault while trying to copy data from user space. In those cases, the code performs a `goto out`, ensuring that the proper cleanup is done.

9. Sculc is a cut-down version of the scull module that implements only the bare device. Unlike scull, which uses `kmalloc`, sculc uses memory caches. The size of the quantum can be modified at compile time and at load time, but not at runtime—that would require creating a new memory cache, and we didn't want to deal with these unneeded details.

Illustrate the technique used to solve this scenario.

`sculc` is a complete example that can be used to try out the slab allocator. It differs from `scull` only in a few lines of code. First, we must declare our own slab cache:

```
/* declare one cache pointer: use it for all devices */
```

```
kmem_cache_t *sculc_cache;
```

The creation of the slab cache is handled (at module load time) in this way:

```
/* sculc_init: create a cache for our quanta */
```

```
sculc_cache = kmem_cache_create("sculc", sculc_quantum,
    0, SLAB_HWCACHE_ALIGN, NULL, NULL); /* no ctor/dtor */
```

```
if (!sculc_cache) {
    sculc_cleanup( );
    return -ENOMEM;
```

```
}
```

This is how it allocates memory quanta:

```
/* Allocate a quantum using the memory cache */
```

```
if (!dptr->data[s_pos]) {
    dptr->data[s_pos] = kmem_cache_alloc(sculc_cache, GFP_KERNEL);
    if (!dptr->data[s_pos])
        goto nomem;
    memset(dptr->data[s_pos], 0, sculc_quantum);
```

```
}
```

And these lines release memory:

```
for (i = 0; i < qset; i++)
```

```
if (dptr->data[i])
```

```
    kmem_cache_free(scullc_cache, dptr->data[i]);
```

Finally, at module unload time, we have to return the cache to the system:

```
/* scullc_cleanup: release the cache of our quanta */
```

```
if (scullc_cache)
```

```
    kmem_cache_destroy(scullc_cache);
```

The main differences in passing from scull to scullc are a slight speed improvement and better memory use. Since quanta are allocated from a pool of memory fragments of exactly the right size, their placement in memory is as dense as possible, as opposed to scull quanta, which bring in an unpredictable memory fragmentation.

10. Compare between `__get_free_pages()` and `kmalloc()` in terms of memory allocations, granularity, physical contiguity and performance.

Factor	<code>kmalloc()</code>	<code>__get_free_pages()</code>
Memory Allocation Granularity	Allocates memory in power-of-two size classes managed by the slab allocator (typically up to 128 KB). Basically allocates memory from a set of fixed size memory pools managed by the slab allocator. Granularity is based on object size, rounded up to next cache size (e.g., 32B, 64B, 128B).	Allocates contiguous pages in blocks of size $\text{PAGE_SIZE} \times 2^{\text{order}}$ (e.g., order=0 → 4 KB, order=2 → 16 KB). Returns a pointer to the first byte of a memory area that is several physically contiguous pages long. Granularity is in units of pages rather than objects.
Physical Contiguity	Returns physically contiguous memory suitable for DMA. Allocation is fulfilled from pre-defined slab caches.	Also returns physically contiguous pages, directly from the buddy system allocator. Larger contiguous regions are harder to allocate under memory pressure.
Performance	Very fast — uses slab caches, so allocations are often $O(1)$ and reuse pre-allocated memory chunks. Lower fragmentation for small objects.	Slightly slower than <code>kmalloc()</code> because it works directly with the page allocator (buddy system), but has less internal fragmentation. Performance decreases for higher orders due to fragmentation and search overhead.

Preferable When	- Small to medium allocations (up to ~128 KB). - Frequently allocated/free kernel buffers. - When cache alignment and reuse matter (e.g., DMA buffers, small data structures).	- When you need multiple contiguous pages (e.g., <code>mmap()</code> buffers, page-aligned arrays). - For page-based allocations (network ring buffers, block I/O buffers). - When the memory will be directly mapped into user space or used for low-level memory manipulation.
-----------------	--	--

11. Draw a neat diagram for the peripheral interface for running digital I/O sample code on a computer.

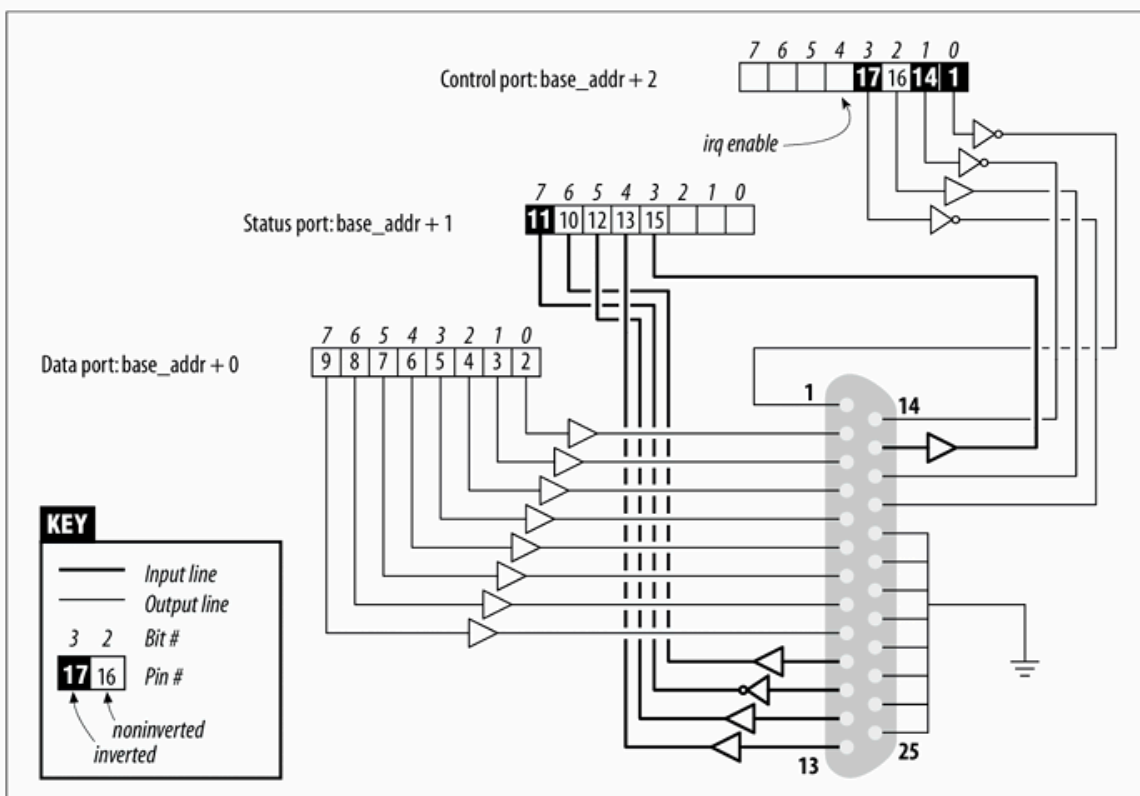


Figure 9-1. The pinout of the parallel port

12. Discuss the difference between I/O port-mapped and memory-mapped I/O.

Every peripheral device is controlled by writing and reading its registers. Most of the time a device has several registers, and they are accessed at consecutive addresses, either in the memory address space or in the I/O address space.

Memory Address Space → called I/O Memory

Separate I/O Address Space → called I/O Ports

At the hardware level, there is no real distinction between memory regions and I/O regions. Both are accessed using electrical signals placed on the address bus and control bus (such as read and write signals), and by transferring data through the data bus.

While some CPU manufacturers implement a single address space in their chips, others decided that peripheral devices are different from memory and therefore, deserve a separate address space. Some processors (most notably the x86 family) have separate read and write electrical lines for I/O ports and special CPU instructions to access ports.

Because peripheral devices are built to fit a peripheral bus, and the most popular I/O buses are modelled on the personal computer, even processors that do not have a separate address space for I/O ports must fake reading and writing I/O ports when accessing some peripheral devices, usually by means of external chipsets or extra circuitry in the CPU core.

Linux defines and supports the concept of I/O ports across all the hardware platforms it runs on, even on systems where the CPU only implements a single, unified address space.

Even if a system's peripheral bus supports a separate address space for I/O ports, not every device uses it.

Older devices, like those on the ISA bus, usually use I/O ports to access their registers.

However, modern devices, such as those on the PCI bus, usually store their registers in memory address space this is called I/O memory.

This I/O memory method is preferred because:

1. It doesn't need special CPU instructions (IN and OUT).
2. The CPU can access memory faster and more efficiently.
3. Compilers can optimize code better when using normal memory operations.

Useful table for comparison:

Factor	Port-Mapped I/O (PIO)	Memory-Mapped I/O (MMIO)
Driver Performance	- Slightly slower, since access is done through special instructions (inb, outb, inw, etc.) that may cause additional CPU overhead. - Not easily optimized by CPU caching or pipelining.	- Generally faster because device registers are mapped into the system's memory space and accessed like normal memory. - CPU can pipeline reads/writes; better for high-speed devices.

Portability	- Less portable — available mainly on x86 and a few other architectures that support separate I/O port spaces. - Other architectures (ARM, MIPS) typically lack port I/O instructions, requiring emulation.	- Highly portable — used on all modern architectures including x86, ARM, PowerPC, RISC-V. - Accessed via standard memory operations or kernel I/O accessors (readl(), writel()).
Hardware Interaction	- Devices are accessed via a separate I/O address space (e.g., ports 0x378–0x37F for parallel ports). - Requires inb()/outb() functions to read or write registers. - Does not require mapping device memory into kernel address space.	- Device registers or buffers are mapped into physical memory addresses. - Drivers must reserve the region (request_mem_region()) and map it using ioremap(). - Accessed using standard kernel memory I/O functions (readl, writel, memcpy_toio).
Example of Appropriate Use	- Legacy ISA/parallel/serial ports on x86 systems. Example: Accessing a parallel port (0x378) using: <pre>outb(value, 0x378);</pre>	- Modern PCI/SoC devices such as network cards, graphics cards, and storage controllers. Example: Mapping device registers with: <pre>base = ioremap(phys_addr, size); writel(val, base + REG_CTRL);</pre>

13. Discuss the portability issue in rewriting the data type between 32-bit and 64-bit architectures in the kernel.

1. Standard C types do not have fixed sizes. 32-bit systems have both int and long as 32b but 64 bit systems have int as 32 bit and long and pointers as 64 bit.
2. Pointer handling: Pointers stored in int or u32 on 32b platform may get truncated in 64b platform. Kernel/Linux recommends to always use unsigned long instead, as it is guaranteed to be of same size as pointer.
3. Byte order/Endianness: Byte order may not be the same across systems. Explicit conversion of ntohs/htons might be needed.
4. Data alignment: Compilers often reorder the fields for memory optimisation. This needs to be taken care of by explicit field ordering and padded data.
5. I/O and printed O/P: size_t, ssize_t etc differ in width and printing them directly with fixed format specifiers like %u or %lu may lead to warnings or incorrect o/p.

14. Give the difference between shared memory model and the message passing model.

Shared Memory	Message Passing
Processes share a common memory region and interact with shared/common variables.	Exchange data through explicit message passing and send/receive.
Fast communication.	Slower due to copying and kernel mediation.
Requires synchronisation using locks to prevent race conditions.	No shared data, synchronisation is simpler.
Used extensively inside the kernel, requires locking primitives.	Used for inter-process communication via pipes, sockets, queues.
Does not work well for isolated processes/distributed systems.	Works well for isolated processes/distributed systems.
Ex: OpenMP, Pthreads library	Ex: MPI

15. Explain the interaction of VFS with file system driver when a process performs read() and write() system call.

— not that sure about this answer — taken from page 342 of understanding linux kernel.

The read or write operation always takes place at the file offset specified by the current file pointer (field `f_pos` of the file object). Both system calls update the file pointer by adding the number of transferred bytes to it.

In short, both `sys_read()` (the `read()`'s service routine) and `sys_write()` (the `write()`'s service routine) perform almost the same steps:

1. Invokes `fget()` to derive from `fd` the address file of the corresponding file object and increments the usage counter `file->f_count`.
2. Checks whether the flags in `file->f_mode` allow the requested access (read or write operation).
3. Invokes `locks_verify_area()` to check whether there are mandatory locks for the file portion to be accessed (see Section 12.6 later in this chapter).
4. If executing a write operation, acquires the `i_sem` semaphore included in the inode object. This semaphore forbids a process to write into the file while another process is flushing to disk buffers relative to the same file (see Section 14.1.5 in Chapter 14). It also forbids two processes to write into the same file at the same time. Notice that, unless the `O_APPEND` flag is set, POSIX does not require serialized file accesses: if a programmer wants exclusive access to a file, he must use a file lock (see next section). Thus, it is possible that a process is reading from a file while another process is writing to it.
5. Invokes either `file->f_op->read` or `file->f_op->write` to transfer the data. Both functions return the number of bytes that were actually transferred. As a side effect, the file pointer is properly updated.
6. Invokes `fput()` to decrement the usage counter `file->f_count`.
7. Returns the number of bytes actually transferred.